

Technical Solution Document — Real Connections Feed

Author: Kundan Yadav

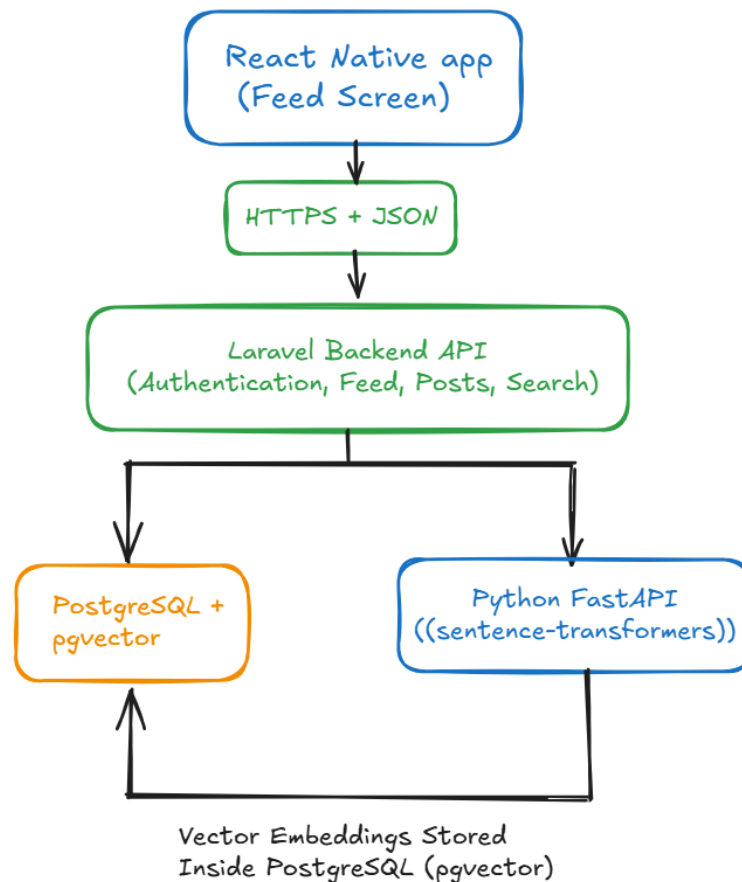
Project: Guised Up — Full-Stack Developer Assessment

1. Overview

This document outlines the design of the **Real Connections Feed**, a personalized feed that prioritizes authenticity, meaningful relationships, and semantic relevance instead of traditional engagement metrics like likes and shares. The system uses **PostgreSQL** for structured data, **pgvector** for semantic search and recommendations, and exposes a **Laravel API** that powers a **React Native** application.

The main idea is simple: help users discover content from people they genuinely connect with, rather than promoting the most popular or attention-grabbing posts.

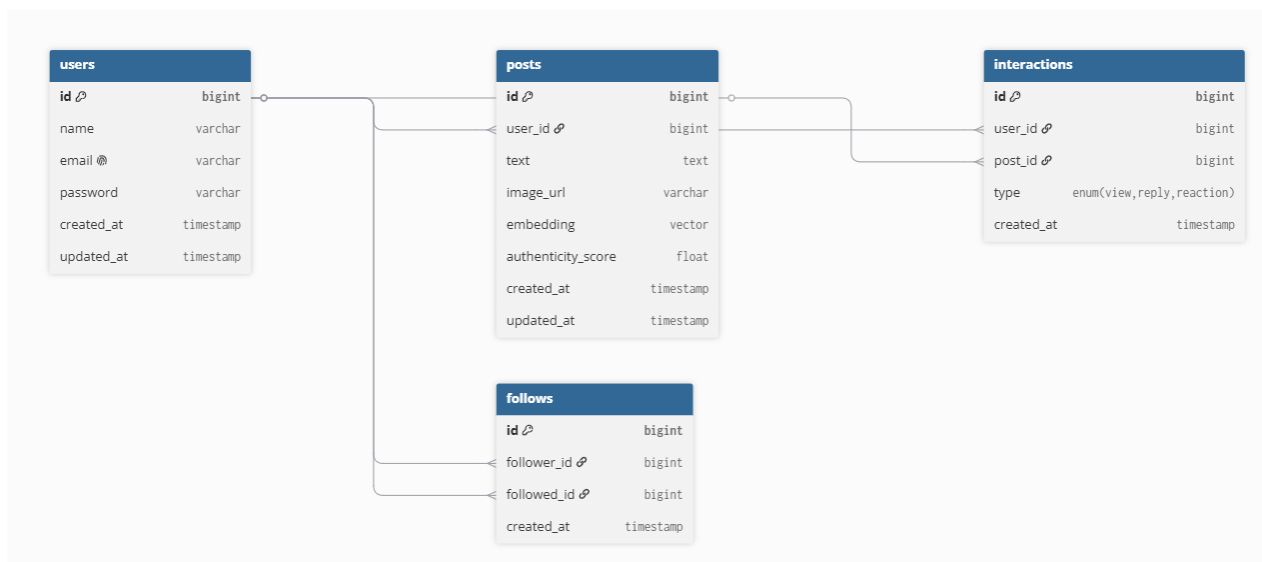
2. System Architecture



Workflow:

1. A user creates a new post by sending a request to `POST /api/posts`.
2. After receiving the post, Laravel forwards the post text to the Python embedding service, which generates and returns a vector embedding.
3. Laravel stores both the post data and its generated embedding in PostgreSQL using the `pgvector` extension.
4. When the user requests `GET /api/feed`, Laravel retrieves candidate posts, calculates a ranking score based on authenticity, relationship depth, semantic similarity, and time decay, and returns a personalized paginated feed.
5. For `GET /api/search`, the search query is converted into an embedding, and PostgreSQL uses cosine similarity on the stored vectors to return the most semantically relevant posts.

3.Database Design



Database Schema Overview

- **Users:** Stores user information such as name, email, and authentication details.
- **Posts:** Stores user-created posts along with the post content, optional image URL, authenticity score, and vector embedding used for semantic search.
- **Interactions:** Records user activities such as views, replies, and reactions. These interactions help calculate the relationship depth between users.
- **Follows:** Maintains the follower–following relationships between users, allowing the platform to understand social connections.

Relationships

- **One User → Many Posts:** A user can create multiple posts.
- **One User → Many Interactions:** A user can interact with multiple posts.
- **One Post → Many Interactions:** A post can receive multiple views, replies, and reactions.

- **Users ↔ Users (Follows):** The follows table represents a many-to-many relationship where users can follow other users.

Indexes

- **posts(created_at)** – Speeds up fetching recent posts for the feed.
- **interactions(user_id, post_id)** – Optimizes relationship depth calculations.
- **interactions(created_at)** – Improves queries for recent user activity.
- **posts(embedding) (IVFFlat Index)** – Enables efficient vector similarity search using pgvector.

4. Vector DB Choice: pgvector

For this project, I chose **pgvector**, a PostgreSQL extension, instead of a dedicated vector database like **Pinecone**, **Weaviate**, or **Qdrant**.

- **Single database for everything:** Posts, user interactions, and vector embeddings are all stored in PostgreSQL. This eliminates the need to synchronize data across multiple databases and makes queries combining semantic similarity and relationship data much simpler.
- **Easy to set up and manage:** Since this is a one-day assessment, using pgvector avoids the overhead of deploying and maintaining an additional vector database service.
- **Well suited for this project's scale:** With the **IVFFlat** index, pgvector provides efficient approximate nearest-neighbor search, which is more than sufficient for the expected dataset size.
- **Easy to scale in the future:** If the application grows to millions of posts, the embedding service is already separated from the main application, making it straightforward to migrate to a dedicated vector database like Qdrant without major architectural changes.

5. API Design

1. POST /api/posts

Purpose:

Creates a new post and generates a vector embedding for the post content before storing it in the database.

Authentication:

Bearer Token (Laravel Sanctum)

Request Body:

```
{
  "text": "Exploring the mountains this weekend!",
  "image_url": "https://example.com/image.jpg"
}
```

Success Response (201 Created):

```
{
  "status": "success",
  "message": "Post created successfully.",
  "data": {
    "id": 12,
    "user_id": 1,
    "text": "Exploring the mountains this weekend!",
    "image_url": "https://example.com/image.jpg",
    "created_at": "2026-07-09T10:30:00Z"
  }
}
```

2. GET /api/feed?page=1

Purpose:

Returns a personalized feed ranked using authenticity, relationship depth, semantic similarity, and time decay.

Authentication:

Bearer Token (Laravel Sanctum)

Query Parameter:

- `page` – Page number for paginated results.

Success Response (200 OK):

```
{
  "current_page": 1,
  "per_page": 20,
  "data": [
    {
      "id": 12,
      "username": "Tanmay",
      "text": "Exploring the mountains this weekend!",
      "image_url": "https://example.com/image.jpg",
      "score": 0.87,
      "created_at": "2026-07-09T10:30:00Z"
    }
  ],
  "next_page": 2
}
```

3. GET /api/search?q={query}

Purpose:

Performs semantic search over posts using vector embeddings and returns the most relevant results based on cosine similarity.

Authentication:

Bearer Token (Laravel Sanctum)

Query Parameter:

- `q` – Natural language search query.

Example Request:

```
GET /api/search?q=travel stories
```

Success Response (200 OK):

```
{
  "query": "travel stories",
  "results": [
    {
      "id": 18,
      "username": "Rahul",
      "text": "My unforgettable trip to Ladakh...",
      "similarity_score": 0.94
    }
  ]
}
```

4. POST `/api/interactions`

Purpose:

Logs user interactions such as views, replies, or reactions. These interactions are later used to calculate relationship depth for feed ranking.

Authentication:

Bearer Token (Laravel Sanctum)

Request Body:

```
{
  "post_id": 12,
  "type": "reaction"
}
```

Supported Interaction Types:

- `view`
- `reply`
- `reaction`

Success Response (201 Created):

```
{  
  "status": "success",  
  "message": "Interaction logged successfully."  
}
```

6. Feed Ranking Algorithm

Overview

The **Real Connections Feed** ranks posts using four key signals instead of traditional engagement metrics like likes or shares. Each candidate post is assigned a score based on how authentic it is, how closely connected the author is to the viewer, how relevant the content is to the user's interests, and how recently the post was created.

Ranking Signals

1. Authenticity Score (30%)

Posts that appear more genuine are given a higher score. This can be estimated using simple heuristics such as natural writing style, meaningful text content, fewer excessive hashtags, and the absence of heavy filters or promotional content.

2. Relationship Depth (30%)

Posts from users with whom the viewer frequently interacts are prioritized. This score is calculated using previous interactions such as views, replies, and reactions rather than simply checking whether the users follow each other.

3. Semantic Similarity (30%)

Each post is converted into a vector embedding. The system compares this embedding with the user's interest vector (generated from their previous posts and interactions) using cosine similarity to determine how relevant the content is.

4. Time Decay (10%)

Newer posts receive a slight advantage using an exponential decay function. This ensures fresh content is promoted while still allowing highly relevant older posts to appear in the feed.

Feed Ranking Formula

Final Score =
0.30 × Authenticity Score
+ 0.30 × Relationship Depth

```
+ 0.30 × Semantic Similarity
+ 0.10 × Time Decay
```

Feed Ranking Pseudocode

```
function score(post, viewingUser):

    authenticity = post.authenticityScore

    relationship = normalize(
        interactionCount(viewingUser, post.author)
    )

    similarity = cosineSimilarity(
        viewingUser.interestVector,
        post.embedding
    )

    hoursOld = hoursSince(post.createdAt)

    timeDecay = exp(-hoursOld / 48)

    return (
        0.30 * authenticity +
        0.30 * relationship +
        0.30 * similarity +
        0.10 * timeDecay
    )

feed = sort(posts by score DESC)
```

Sample Feed Response

```
{
  "data": [
    {
      "id": 12,
      "user": {
        "id": 3,
        "name": "Rahul"
      },
      "text": "Exploring the mountains this weekend!",
      "score": 0.87,
      "created_at": "2026-07-09T10:30:00Z"
    }
  ],
  "next_page": 2
}
```

Notes

- The ranking weights are configurable and can be adjusted based on user feedback or A/B testing.

- The current values are chosen to demonstrate the ranking strategy for this assessment and are not based on production data.
 - The algorithm is designed to balance authenticity, relevance, and freshness while encouraging meaningful social interactions over popularity.
-

7. AI Tool Usage

AI-assisted development was used throughout this project to improve productivity while ensuring that all architectural and design decisions were made after careful evaluation.

Claude

- Assisted in brainstorming the overall system architecture and database schema.
- Helped draft and refine the Technical Solution Document (TSD).
- Provided guidance for designing the feed ranking algorithm and API structure.

Github Copilot

- Used to generate Laravel boilerplate code, including controllers, models, migrations, and API routes.
- Assisted with code completion and refactoring during backend development.
- Helped speed up repetitive coding tasks while maintaining code quality.

AI Assistance for Vector Search

- Used AI to verify pgvector configuration, indexing strategies, and cosine similarity query syntax.
- Assisted in understanding best practices for integrating semantic search with PostgreSQL.

Overall Contribution

AI tools significantly reduced development time by handling repetitive coding tasks, documentation, and syntax-related issues. However, all major technical decisions—including the choice of pgvector, database schema, API design, and feed ranking strategy—were evaluated and finalized manually.

8. Trade-offs & Assumptions

Since this project was designed to be completed within a limited timeframe, a few practical assumptions and simplifications were made.

Embedding Generation

A lightweight embedding solution (or mocked embedding if API access is unavailable) is assumed for this assessment. In a production environment, this can be replaced with models such as **Sentence Transformers** or **OpenAI Embeddings** without changing the overall architecture.

Ranking Weights

The weights assigned to authenticity, relationship depth, semantic similarity, and time decay are illustrative. In a real-world application, these values would be optimized using user feedback, analytics, and A/B testing.

Relationship Depth

Relationship strength is calculated using the total number of user interactions. Different interaction types (such as replies being more valuable than views) are treated equally for simplicity.

Authenticity Scoring

Authenticity is estimated using simple heuristics, such as meaningful text content, reasonable post length, and limited hashtag usage. A production system could use a machine learning model to provide more accurate authenticity predictions.

User Interest Vector

A user's interest profile is assumed to be the average of embeddings from their recent posts and interactions. This serves as a simple approximation of user preferences and can later be replaced with a more advanced recommendation model.